

Laravel Project

1.Laravel Installation(<https://laravel.com/docs/11.x/installation>)

- composer global require laravel/installer
- laravel new example-app
- cd example-app
- npm install
- npm run build
- composer run dev

2.Laravel Breeze(<https://laravel.com/docs/11.x/starter-kits>)

- composer require laravel/breeze --dev
- php artisan breeze:install
- php artisan migrate
- npm install
- npm run dev

3.Laravel adminlte(<https://github.com/jeroennoten/Laravel-AdminLTE/wiki/Installation>)

- composer require jeroennoten/laravel-adminlte
- php artisan adminlte:install
- composer require laravel/ui
- php artisan ui bootstrap --auth
- php artisan adminlte:install --only=auth_views
- Usage(<https://github.com/jeroennoten/Laravel-AdminLTE/wiki/Usage>)

➤ Paste home.blade.php

```
@extends('adminlte::page')
@section('title', 'Dashboard')
@section('content_header')
    <h1>Dashboard</h1>
@stop
@section('content')
    <p>Welcome to this beautiful admin panel.</p>
@stop
@section('css')
    {{-- Add here extra stylesheets --}}
```

```
{{-- <link rel="stylesheet" href="/css/admin_custom.css"> --}}
```

```
@stop
```

```
@section('js')
```

```
<script> console.log("Hi, I'm using the Laravel-AdminLTE  
package!"); </script>
```

```
@stop
```

➤ edit config/adminlte.php

- Change title
- Change logo
- Change layout_fixed_sidebar =>true
layout_fixed_Navbar=> true
layout_fixed_Footer =>true
- Sidebar_collape =>true
Sidebar_collape_auto_size =>true
Sidebar_collape_remember_no_transition=>true
- Use_route_url=> false
Dashboard_url =>home
- Menu=>
//Navbar item
Sidebar item
//blog

```
['text' => 'Dashboard',  
'url' => 'home',  
'icon' => 'fas fa-th text-yellow',
```

```
],
```

```
[  
'text' => 'User Management',  
'icon' => 'fa fa fa- fa-users text-teal',  
'submenu' => [  
    [  
        'text' => 'Users',  
        'url' => 'users',  
    ],  
],
```

```
],
```

```
[  
'text' => 'Master Data',  
'icon' => 'fa fa fa- fa-cog',  
'submenu' => [  
    [  
        'text' => 'Rank',  
        'url' => 'ranks',  
    ],  
],
```

```

        'text' => 'Location',
        'url' => 'locations',

    ],
    [
        'text' => 'Unit',
        'url' => 'units',

    ],

],
],

```

4. Install Spatie(<https://spatie.be/docs/laravel-permission/v6/installation-laravel>)

- composer require spatie/laravel-permission
- php artisan vendor:publish --
provider="Spatie\Permission\PermissionServicePr
ovider"

```
php artisan optimize:clear
```

or

```
php artisan config:clear
```

- php artisan migrate
- Add the necessary trait to your User model:
// The User model requires this trait
use Spatie\Permission\Traits\HasRoles;
use HasRoles;
- php artisan make:seeder RoleAndPermissionSeeder

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Spatie\Permission\Models\Role;
use Spatie\Permission\Models\Permission;
use App\Models\User;

class RoleAndPermissionSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        // Create Permissions
        Permission::create(['name' => 'view_users']);
        Permission::create(['name' => 'create_users']);
        Permission::create(['name' => 'edit_users']);
        Permission::create(['name' => 'delete_users']);
        Permission::create(['name' => 'view_ranks']);
        Permission::create(['name' => 'create_ranks']);
    }
}
```

```

Permission::create(['name' => 'edit_ranks']);
Permission::create(['name' => 'delete_ranks']);
Permission::create(['name' => 'view_locations']);
Permission::create(['name' => 'create_locations']);
Permission::create(['name' => 'edit_locations']);
Permission::create(['name' => 'delete_locations']);
Permission::create(['name' => 'view_units']);
Permission::create(['name' => 'create_units']);
Permission::create(['name' => 'edit_units']);
Permission::create(['name' => 'delete_units']);

// Create Roles
$adminRole = Role::create(['name' => 'admin']);
$officerRole = Role::create(['name' => 'officer']);
$operatorRole = Role::create(['name' => 'operator']);

// Assign Permissions to Admin
$adminRole->givePermissionTo(Permission::all());

// Assign Permissions to Officer
$officerRole->givePermissionTo([
    'view_users',
    'create_users',
    'edit_users',
    'view_ranks',
    'create_ranks',
    'edit_ranks',
    'view_locations',
    'create_locations',
    'edit_locations',
    'view_units',
    'create_units',
    'edit_units',
]);
// Assign Permissions to Operator
$operatorRole->givePermissionTo([
    'view_users',
    'view_ranks',
    'view_locations',
    'view_units',
]);
$user = User::find(1); // Find user with ID 1
$user->assignRole('admin'); // Assign 'admin' role

$user = User::find(2); // Find user with ID 2
$user->assignRole('officer'); // Assign 'officer' role

$user = User::find(3); // Find user with ID 3
$user->assignRole('operator'); // Assign 'operator' role
}

```

- php artisan db:seed --class=RoleAndPermissionSeeder
- php artisan tinker

```

$user = User::find(1);
$user->assignRole('admin');
$user = User::find(2);

```

```
$user->assignRole('officer');  
$user = User::find(3);  
$user->assignRole('operator');
```

- Models\User.php

```
public function role()  
{  
    return $this->belongsTo(Role::class, 'role_id');  
}
```

- php artisan make:model Role

```
public function permission(){  
    return $this->belongsToMany(Permission::class);  
}  
public function user(){  
    return $this->hasMany(User::class);  
}  
}
```

- php artisan make:model Permission

```
public function role(){  
    return $this->belongsToMany(Role::class, 'role_id');  
}
```

- php artisan make:controller UserController

```
<?php  
  
namespace App\Http\Controllers;  
use App\Models\User;  
use App\Models\Location; // Import Location Model  
use App\Models\Role;  
use App\Models\Rank;  
use App\Models\Unit;  
use App\DataTables\UsersDataTable;  
use Illuminate\Support\Facades\Hash;  
use Illuminate\Http\Request;  
  
class UserController extends Controller  
{  
    public function __construct()  
    {  
        $this->middleware('permission:view_users')->only('index', 'show');  
        $this->middleware('permission:create_users')->only('create', 'store');  
  
        $this->middleware('permission:edit_users')->only('edit', 'update');  
        $this->middleware('permission:delete_users')->only('destroy');  
    }  
    public function index(UsersDataTable $dataTable)  
    {  
        return $dataTable->render('users.index');  
    }  
    public function create()  
    {  
        $location = Location::all();  
        $role = Role::all();  
        $rank = Rank::all();  
        $unit = Unit::all();  
    }  
}
```

```

        return view('users.create', compact('location','role','rank','unit'));
    }
    public function store(Request $request)
    {
        $validated = $request->validate([
            'service_no' => 'required|string|max:255',
            'name' => 'required|string|max:255',
            'email' => 'required|email|unique:users,email',
            'password' => 'required|string|min:8',
            'location_id' => ['required', 'numeric'],
            'rank_id' => ['required', 'numeric'],
            'unit_id' => ['required', 'numeric'],
            'password' => ['required', 'string', 'min:8'], // Optional field
            'role_id' => ['required'], // Validate role input
        ]);
        $user = User::create($validated);
        $user->password = Hash::make($request->password);
        $user->active = 1;
        $user->save();

        $role = Role::findOrFail($request->input('role_id'));

        // Sync the roles for the user (in case role is updated)

        $user->syncRoles([$role->name]);
        $user->assignRole($role->name);
        return redirect()->route('users.index');
    }
    public function show(User $user)
    {
        return view('users.show', compact('user'));
    }
    public function edit(User $user)
    {
        $location = Location::all();
        $role = Role::all();
        $rank = Rank::all();
        $unit = Unit::all();
        return view('users.edit', compact('user','location','role','rank','unit'));
    }
    public function update(Request $request, User $user)
    {
        $user_detail = $request->validate([
            'service_no' => ['required', 'string'],
            'name' => 'required|string|max:255',
            'email' => 'required|string|max:255',
            'location_id' => ['required', 'numeric'],
            'rank_id' => ['required', 'numeric'],
            'unit_id' => ['required', 'numeric'],
            'password' => 'required|string|min:8', // Optional field
            'role_id' => ['required'], // Validate role input
        ]);
        $user->update($user_detail);
        $role = Role::findOrFail($request->input('role_id'));
        // Sync the roles for the user (in case role is updated)
        $user->syncRoles([$role->name]);
        $user->assignRole($role->name);

        $user->syncPermissions($request->input('permissions'));
        // $user->update($request->all());
    }

```

```

        return redirect()->route('users.index');
    }
    public function destroy(User $user)
    {
        $user->delete();
        return redirect()->route('users.index');
    }
}

```

- Bootstrap/app.php

```

->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'role' => \Spatie\Permission\Middleware\RoleMiddleware::class,
        'permission' => \Spatie\Permission\Middleware\PermissionMiddleware::class,
        'role_or_permission' =>
\Spatie\Permission\Middleware\RoleOrPermissionMiddleware::class,
    ]);
})

```

- Web.php

```

<?php

use App\Http\Controllers\ProfileController;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\LocationController;
use App\Http\Controllers\RankController;
use App\Http\Controllers\UnitController;
use App\Http\Controllers\UserController;

Route::get('/', function () {
    return view('auth.login');
});
Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
});
Route::middleware('auth')->group(function () {
    Route::resource('locations', LocationController::class);
    Route::resource('ranks', RankController::class);
    Route::resource('units', UnitController::class);
    Route::resource('users', UserController::class);
});
require __DIR__.'/auth.php';

Auth::routes();

Route::get('/home', [App\Http\Controllers\HomeController::class, 'index'])->name('home');

```

5.create migration

- Php artisan make:migration create_user_table
- Edit migration

```

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

class CreateUsersTable extends Migration
{
    public function up()
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('service_no');
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->rememberToken();
            $table->tinyInteger('active')->default(0);
            $table->dateTime('active_date_time')->nullable();
            $table->unsignedBigInteger('role_id')->nullable(); // Use appropriate type
            $table->foreign('role_id')->references('id')->on('roles')->onDelete('cascade');
            $table->unsignedBigInteger('location_id')->nullable(); // Use appropriate type
            $table->foreign('location_id')->references('id')->on('locations')->onDelete('cascade'); // Define the action on delete
            $table->unsignedBigInteger('rank_id')->nullable(); // Use appropriate type
            $table->foreign('rank_id')->references('id')->on('ranks')->onDelete('cascade'); // Define the action on delete
            $table->unsignedBigInteger('unit_id')->nullable(); // Use appropriate type
            $table->foreign('unit_id')->references('id')->on('units')->onDelete('cascade'); // Define the action on delete
            $table->timestamps();
        });
    }

    public function down()
    {
        Schema::dropIfExists('users');
    }
}

```

- php artisan migrate

6.crud

- Create.blade.php

```

@extends('adminlte::page')

@section('content')
<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-7">
            @if ($errors->any())
            <div class="alert alert-danger">
                <ul>
                    @foreach ($errors->all() as $error)
                        <li>{{ $error }}</li>
                    @endforeach
                </ul>
            </div>
            @endif
        </div>
    </div>
</div>

```



```

        </ul>
    </div>
@endif

    @if (session('status'))
        <div class="alert alert-success">{{ session('status') }}</div>
    @endif
    <div class="card">
        <div class="card-header">{{ __(' Create User') }}</div>
        <div class="card-body">
            <form action="{{ route('users.store') }}" method="POST">
                @csrf
                <div class="mb-3">
                    <label for="">Service No:</label>
                    <input type="text" name="service_no" required class="form-
control"/>

                </div>
                <div class="mb-3">
                    <label for="">Name:</label>
                    <input type="text" name="name" required class="form-control"/>
                </div>
                <div class="mb-3">
                    <label for="">Email: </label>
                    <input type="text" name="email" required class="form-control"/>
                </div>
                <div class="mb-3">
                    <label for="">Rank: </label>
                    <select name="rank_id" id="rank_id" class="form-control" required>
                        @foreach ($rank as $rank)
                            <option value="{{ $rank->id }}">{{ $rank->name }}</option>
                        @endforeach
                    </select>
                </div>
                <div class="mb-3">
                    <label for="">Unit: </label>
                    <select name="unit_id" id="unit_id" class="form-control" required>
                        @foreach ($unit as $unit)
                            <option value="{{ $unit->id }}">{{ $unit->name }}</option>
                        @endforeach
                    </select>
                </div>
                <div class="mb-3">
                    <label for="">Location: </label>
                    <select name="location_id" id="location_id" class="form-control"
required>
                        @foreach ($location as $location)
                            <option value="{{ $location->id }}">{{ $location->name
}}</option>

                        @endforeach
                    </select>
                </div>
                <div class="mb-3">
                    <label for="">Role: </label>
                    <select name="role_id" id="role_id" class="form-control" required>
                        @foreach ($role as $role)
                            <option value="{{ $role->id }}">{{ $role->name }}</option>
                        @endforeach
                    </select>
                </div>
                <div class="mb-3">

```

```

        <label for="">Password: </label>
        <input type="text" name="password" required class="form-control"/>
    </div>

    <div class="mb-3">
        <button type="submit" class="btn btn-primary">Save</button>
    </div>
</form>

</div>
</div>
</div>
</div>
</div>

```

- Edit.blade.php

```

@extends('adminlte::page')

@section('content')
<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-7">
            @if ($errors->any())
            <div class="alert alert-danger">
                <ul>
                    @foreach ($errors->all() as $error)
                        <li>{{ $error }}</li>
                    @endforeach
                </ul>
            </div>
            @endif
            @if (session('status'))
                <div class="alert alert-success">{{ session('status') }}</div>
            @endif
            <div class="card">
                <div class="card-header">{{ __('Edit User') }}</div>
                <div class="card-body">
                    <form action="{{ route('users.update', $user->id) }}" method="POST">
                        @csrf
                        @method('PUT')
                        <div class="mb-3">
                            <label for="">Service No:</label>
                            <input type="text" name="service_no" id="service_no" class="form-control" value="{{ old('service_no', $user->service_no) }}" required>
                        </div>
                        <div class="mb-3">
                            <label for="">Name:</label>
                            <input type="text" name="name" id="name" class="form-control" value="{{ old('name', $user->name) }}" required>
                        </div>
                        <div class="mb-3">
                            <label for="">Email:</label>
                            <input type="text" name="email" id="email" class="form-control" value="{{ old('email', $user->email) }}" required>
                        </div>
                        <div class="mb-3">
                            <label for="">Rank:</label>
                            <select name="rank_id" id="rank_id" class="form-control" required>

```

```

        @foreach ($rank as $rank)
            <option value="{{ $rank->id }}">{{ $rank->name
    }}</option>

        @endforeach
    </select>
</div>
<div class="mb-3">
    <label for="">Unit: </label>
    <select name="unit_id" id="unit_id" class="form-control"
required>

        @foreach ($unit as $unit)
            <option value="{{ $unit->id }}">{{ $unit->name
    }}</option>

        @endforeach
    </select>
</div>
<div class="mb-3">
    <label for="">Location: </label>
    <select name="location_id" id="location_id" class="form-
control" required>

        @foreach ($location as $location)
            <option value="{{ $location->id }}">{{ $location->name
    }}</option>

        @endforeach
    </select>
</div>
<div class="mb-3">
    <label for="">Role: </label>
    <select name="role_id" id="role_id" class="form-control"
required>

        @foreach ($role as $role)
            <option value="{{ $role->id }}">{{ $role->name
    }}</option>

        @endforeach
    </select>
</div>

</div>
<div class="mb-3">
    <label for="">Password: </label>
    <input type="text" name="password" id="password" class="form-
control" value="{{ old('password', $user->password) }}" required>

    </div>

    <div class="mb-3">
        <button type="submit" class="btn btn-primary">Update</button>
    </div>
</form>
</div>
</div>
</div>
</div>
</div>

```

- Index.blade.php

```
@extends('adminlte::page')
```

```

@section('content')
<div class="container mt-5">
  <div class="row justify-content-center">
    <div class="col-md-12">

      @if (session('status'))
        <div class="alert alert-success">{{ session('status') }}</div>
      @endif
      <div class="card mt-3">
        <div class="card-header">{{ __('Users') }}</div>

        <div class="card-body">
          <a href="{{ route('users.create') }}" class="btn btn-primary float-end">Add
New User</a>

          {{-- <table class="table table-bordered table-striped">
            <thead>
              <tr>
                <th>Id</th>
                <th>Service_no</th>
                <th>Name</th>
                <th>Email</th>
                <th>Active</th>
                <th>Role</th>
                <th>Rank</th>
                <th>Location</th>
                <th>Unit</th>
                <th>Created_at</th>
                <th>Updated_at</th>
                <th>Actions</th>
              </tr>
            </thead>
            <tbody>
              @foreach ($users as $user)

                <tr>
                  <td>{{ $user->id }}</td>
                  <td>{{ $user->service_no }}</td>
                  <td>{{ $user->name }}</td>
                  <td>{{ $user->email }}</td>
                  <td>{{ $user->active }}</td>
                  <td>{{ $user->role->name }}</td>
                  <td>{{ $user->ranks->name }}</td>
                  <td>{{ $user->locations->name }}</td>
                  <td>{{ $user->units->name }}</td>
                  <td>{{ $user->created_at }}</td>
                  <td>{{ $user->updated_at }}</td>

                  <td>
                    <a href="{{ route('users.edit', $user->id) }}" class="btn btn-xs btn-
warning" data-toggle="tooltip" title="Edit User" ><i class="fa fa-pen"></i></a>
                    <form action="{{ route('users.destroy', $user->id) }}"
method="POST" style="display:inline;" onsubmit="return confirmDelete()" >
                      @csrf
                      @method('DELETE')
                      <button type="submit" class="btn btn-xs btn-danger"
data-toggle="tooltip" title="Delete User" ><i class="fa fa-trash"></i></button>
                    </form>
                    <a href="{{ route('users.show', $user->id) }}" class="btn
btn-xs btn-info" data-toggle="tooltip" title="Edit User" ><i class="fa fa-eye"></i></a>

```

```

        </td>
    </tr>
    @endforeach
</tbody>
</table> --}}

<div class="table-responsive">
    {{ $dataTable->table() }}
</div>
</div>
</div>
</div>
@endsection
{{-- <script>
    function confirmDelete(){
        return confirm('Are You sure you want to delete this record? This action be
undone.');
```

- Show.blade.php

```

@extends('adminlte::page')

@section('content')
<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-7">

            @if (session('status'))
                <div class="alert alert-success">{{ session('status') }}</div>
            @endif
            <div class="card">
                <div class="card-header"><strong>{{ $user->id }}</strong>
                </div></div>
                <div class="card-body">
                    <ul class="list-group">
                        <li class="list-group-item">
                            <strong>Service No:</strong> {{ $user->service_no }}
                        </li>
                        <li class="list-group-item">
                            <strong>Name:</strong> {{ $user->email }}
                        </li>
                        <li class="list-group-item">
                            <strong>Email:</strong> {{ $user->email }}
                        </li>
                        <li class="list-group-item">
                            <strong>Active:</strong> {{ $user->active }}
                        </li>
                        <li class="list-group-item">
                            <strong>Role:</strong> {{ $user->role->name }}
                        </li>
                        <li class="list-group-item">
                            <strong>Rank:</strong> {{ $user->ranks->name }}
                        </li>
                    </ul>
                </div>
            </div>
        </div>
    </div>
</div>

```

```

        </li>
        <li class="list-group-item">
            <strong>Location:</strong> {{ $user->locations->name }}
        </li>
        <li class="list-group-item">
            <strong>Unit:</strong> {{ $user->units->name }}
        </li>
        <li class="list-group-item">
            <strong>Created At:</strong>
            {{ $user->created_at ? $user->created_at->format('d-m-Y H:i') :
'N/A' }}
        </li>
        <li class="list-group-item">
            <strong>Last Updated:</strong>
            {{ $user->updated_at ? $user->updated_at->format('d-m-Y H:i') :
'N/A' }}
        </li>
    </ul>
</div>
</div>
</div>
</div>
</div>

```

7.Yajra laravel datatables(<https://yajrabox.com/docs/laravel-datatables/12.0>)

- `composer require yajra/laravel-datatables-oracle:"^12.0"`
- `composer require yajra/laravel-datatables:"^12.0"`
- `php artisan vendor:publish --tag=datatables`
- (<https://yajrabox.com/docs/laravel-datatables/12.0/quick-starter>)

```
npm i laravel-datatables-vite --save-dev
```

```
resources/js/app.js
```

```
import 'laravel-datatables-vite';
```

```
resources/sass/app.scss
```

```
// DataTables
@import 'bootstrap-icons/font/bootstrap-icons.css';
@import "datatables.net-bs5/css/dataTables.bootstrap5.min.css";
@import "datatables.net-buttons-bs5/css/buttons.bootstrap5.min.css";
@import 'datatables.net-select-bs5/css/select.bootstrap5.css';
```

```
npm run dev
```

```
php artisan datatables:make Users
```

```
<?php
namespace App\DataTables;
```

```

use App\Models\User;
use Illuminate\Database\Eloquent\Builder as QueryBuilder;
use Yajra\DataTables\EloquentDataTable;
use Yajra\DataTables\Html\Builder as HtmlBuilder;
use Yajra\DataTables\Html/Button;
use Yajra\DataTables\Html\Column;
use Yajra\DataTables\Html/Editor/Editor;
use Yajra\DataTables\Html/Editor/Fields;
use Yajra\DataTables\Services\DataTable;

class UsersDataTable extends DataTable
{
    /**
     * Build the DataTable class.
     *
     * @param QueryBuilder<User> $query Results from query() method.
     */
    public function dataTable(QueryBuilder $query): EloquentDataTable
    {
        return (new EloquentDataTable($query))
            ->addIndexColumn()
            ->addColumn('active', function ($usersdatatable) {
                switch ($usersdatatable->active) {
                    case 0:
                        return '<span class="badge badge-danger">In-Active</span>';
                        break;
                    case 1:
                        return '<span class="badge badge-success">Active</span>';
                        break;
                }
            })
            ->addColumn('role.name', function ($user) {
                return '<span class="badge badge-primary">' . $user->role->name. '</span>';
            })
            ->addColumn('action', function ($usersdatatable) {
                $btn = '<a href="'.route('users.edit',$usersdatatable->id).'" class="btn btn-xs btn-warning" data-toggle="tooltip" title="Edit User" ><i class="fa fa-pen"></i></a> ';
                $btn .= '<form action="'. route('users.destroy', $usersdatatable->id) . '" method="POST" class="d-inline" onsubmit="return confirmDelete()" >
                    ' . csrf_field() . '
                    ' . method_field("DELETE") . '
                    <button type="submit" class="btn bg-danger btn-xs dark:bg-gray-800 dark:hover:bg-gray-700 dark:focus:ring-gray-700" onclick="return confirm(\'Do you need to delete this\');" data-toggle="tooltip" title="Delete">
                        <i class="fa fa-trash-alt"></i></button>
                    </form> </div>';
                $btn .= '<a href="'.route('users.show', $usersdatatable->id).'" class="btn btn-xs btn-info" data-toggle="tooltip" title="View User" ><i class="fa fa-eye"></i></a> ';
                return $btn;
            })
            ->rawColumns(['active', 'action','role.name']);
    }

    /**
     * Get the query source of dataTable.
     *
     * @return QueryBuilder<User>
     */
    public function query(User $model): QueryBuilder
    {

```

```

        return $model->newQuery()->with([
            'role',
            'ranks',
            'locations',
            'units'
        ]);
    }
    /**
     * Optional method if you want to use the html builder.
     */
    public function html(): HtmlBuilder
    {
        return $this->builder()
            ->setTableId('users-table')
            ->columns($this->getColumns())
            ->minifiedAjax()
            ->orderBy(1)
            ->selectStyleSingle()
            ->buttons([
                Button::make('excel'),
                Button::make('csv'),
                Button::make('pdf'),
                Button::make('print'),
                Button::make('reset'),
                Button::make('reload')
            ]);
    }

    /**
     * Get the dataTable columns definition.
     */
    public function getColumns(): array
    {
        return [
            // Column::make('DT_RowIndex')->title('#')->searchable(false)->orderable(false)-
            >width(5),
            Column::make('id'),
            Column::make('service_no')->title('Service No')->data('service_no')-
            >searchable(true),
            Column::make('name')->title('Name')->data('name')->searchable(true),
            Column::make('email')->title('Email')->data('email')->searchable(true),
            Column::make('active')->title('Active')->data('active')->searchable(true),
            Column::make('role.name')->title('Role')->data('role.name')->searchable(true),
            Column::make('ranks.name')->title('Rank')->data('ranks.name')->searchable(true),
            Column::make('locations.name')->title('Location')->data('locations.name')-
            >searchable(true),
            Column::make('units.name')->title('Unit')->data('units.name')->searchable(true),
            Column::computed('action')
                ->exportable(false)
                ->printable(false)
                ->width(80)
                ->addClass('text-center'),
        ];
    }

    /**
     * Get the filename for export.
     */
    protected function filename(): string
    {

```



```

        return 'Users_' . date('YmdHis');
    }
}

```

8.Tricks

- How do I confirm delete?

In index.php

```

@if (session('status'))
    <div class="alert alert-success">{{ session('status') }}</div>
@endif

```

```

<a href="{{ route('users.edit', $user->id) }}" class="btn btn-xs btn-warning" data-
toggle="tooltip" title="Edit User" ><i class="fa fa-pen"></i></a>
    <form action="{{ route('users.destroy', $user->id) }}"
method="POST" style="display:inline;" onsubmit="return confirmDelete()" >
        @csrf
        @method('DELETE')
        <button type="submit" class="btn btn-xs btn-danger"
data-toggle="tooltip" title="Delete User" ><i class="fa fa-trash"></i></button>
    </form>

```

```

@endsection
{{-- <script>
    function confirmDelete(){
        return confirm('Are You sure you want to delete this record? This action be
undone. ');
    }
</script> --}}

```

- How do you get back in after jumping out of the table?

In index.php

```

<div class="table-responsive">
    {{ $dataTable->table() }}
</div>

```

- How to install softdelete?

- Create migration
- Php artisan make:migration add_deleted_at_to_users_table
- In user model

```

<?php
namespace App\Models;

```

```
// use Illuminate\Contracts\Auth\MustVerifyEmail;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Spatie\Permission\Traits\HasRoles;
use Illuminate\Database\Eloquent\SoftDeletes;

class User extends Authenticatable
{
    /** @use HasFactory<\Database\Factories\UserFactory> */
    use HasFactory, Notifiable ,HasRoles, SoftDeletes;
```

- In add_deleted_at_to users_table.php

```
public function up(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->softDeletes();
    });
}
```

- How to make a connection?

In user.php

```
<?php

namespace App\Models;

// use Illuminate\Contracts\Auth\MustVerifyEmail;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Spatie\Permission\Traits\HasRoles;
use Illuminate\Database\Eloquent\SoftDeletes;

class User extends Authenticatable
{
    /** @use HasFactory<\Database\Factories\UserFactory> */
    use HasFactory, Notifiable ,HasRoles, SoftDeletes;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'id',
        'service_no',
        'name',
        'email',
        'email_verified_at',
        'password',
        'remember_token',
        'active_date_time',
        'active',
    ];
```

```

        'created_at',
        'updated_at',
        'location_id',
        'role_id',
        'rank_id',
        'unit_id',
    ];
    /**
     * The attributes that should be hidden for serialization.
     *
     * @var list<string>
     */
    protected $hidden = [
        'password',
        'remember_token',
    ];
    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
        ];
    }
    public function ranks()
    {
        return $this->belongsTo(Rank::class, 'rank_id');
    }
    public function locations()
    {
        return $this->belongsTo(Location::class, 'location_id');
    }
    public function units()
    {
        return $this->belongsTo(Unit::class, 'unit_id');
    }
    public function role()
    {
        return $this->belongsTo(Role::class, 'role_id');
    }
}

```

In controller

```

<?php

namespace App\Http\Controllers;
use App\Models\User;
use App\Models\Location; // Import Location Model
use App\Models\Role;
use App\Models\Rank;
use App\Models\Unit;
use App\DataTables\UsersDataTable;
use Illuminate\Support\Facades\Hash;

```

```

use Illuminate\Http\Request;

class UserController extends Controller
{
    public function __construct()
    {
        $this->middleware('permission:view_users')->only('index', 'show');
        $this->middleware('permission:create_users')->only('create', 'store');
        $this->middleware('permission:edit_users')->only('edit', 'update');
        $this->middleware('permission:delete_users')->only('destroy');
    }

    public function index(UsersDataTable $dataTable)
    {
        return $dataTable->render('users.index');
    }

    public function create()
    {
        $location = Location::all();
        $role = Role::all();
        $rank = Rank::all();
        $unit = Unit::all();
        return view('users.create', compact('location','role','rank','unit'));
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'service_no' => 'required|string|max:255',
            'name' => 'required|string|max:255',
            'email' => 'required|email|unique:users,email',
            'password' => 'required|string|min:8',
            'location_id' => ['required', 'numeric'],
            'rank_id' => ['required', 'numeric'],
            'unit_id' => ['required', 'numeric'],
            'password' => ['required', 'string', 'min:8'], // Optional field
            'role_id' => ['required'], // Validate role input
        ]);

        $user = User::create($validated);
        $user->password = Hash::make($request->password);
        $user->active = 1;
        $user->save();

        $role = Role::findOrFail($request->input('role_id'));

        // Sync the roles for the user (in case role is updated)
        $user->syncRoles([$role->name]);
        $user->assignRole($role->name);

        return redirect()->route('users.index');
    }

    public function show(User $user)
    {
        return view('users.show', compact('user'));
    }

    public function edit(User $user)
    {
        $location = Location::all();
        $role = Role::all();
    }

```

```

$rank = Rank::all();
$unit = Unit::all();
return view('users.edit', compact('user','location','role','rank','unit'));
}
public function update(Request $request, User $user)
{
    $user_detail = $request->validate([
        'service_no' => ['required', 'string'],
        'name' => 'required|string|max:255',
        'email' => 'required|string|max:255',
        'location_id' => ['required', 'numeric'],
        'rank_id' => ['required', 'numeric'],
        'unit_id' => ['required', 'numeric'],
        'password' => 'required|string|min:8', // Optional field
        'role_id' => ['required'], // Validate role input
    ]);
    $user->update($user_detail);
    $role = Role::findOrFail($request->input('role_id'));
    // Sync the roles for the user (in case role is updated)
    $user->syncRoles([$role->name]);
    $user->assignRole($role->name);

    $user->syncPermissions($request->input('permissions'));

    // $user->update($request->all());
    return redirect()->route('users.index');
}
public function destroy(User $user)
{
    $user->delete();
    return redirect()->route('users.index');
}
}

```

In userDataTable

```

public function query(User $model): QueryBuilder
{
    return $model->newQuery()->with([
        'role',
        'ranks',
        'locations',
        'units'
    ]);
}

```

In unit.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;
class Unit extends Model
{
    use SoftDeletes;
    protected $fillable = [
        'name',
    ];
    public function user()
    {

```

```

        return $this->hasmany(user::class);
    }
}

```

In user controller

```

<?php

namespace App\Http\Controllers;
use App\Models\User;
use App\Models\Location; // Import Location Model
use App\Models\Role;
use App\Models\Rank;
use App\Models\Unit;
use App\DataTables\UsersDataTable;
use Illuminate\Support\Facades\Hash;

use Illuminate\Http\Request;

class UserController extends Controller
{
    public function __construct()
    {
        $this->middleware('permission:view_users')->only('index', 'show');
        $this->middleware('permission:create_users')->only('create', 'store');
        $this->middleware('permission:edit_users')->only('edit', 'update');
        $this->middleware('permission:delete_users')->only('destroy');
    }

    public function index(UsersDataTable $dataTable)
    {
        return $dataTable->render('users.index');
    }

    public function create()
    {
        $location = Location::all();
        $role = Role::all();
        $rank = Rank::all();
        $unit = Unit::all();
        return view('users.create', compact('location','role','rank','unit'));
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'service_no' => 'required|string|max:255',
            'name' => 'required|string|max:255',
            'email' => 'required|email|unique:users,email',
            'password' => 'required|string|min:8',
            'location_id' => ['required', 'numeric'],
            'rank_id' => ['required', 'numeric'],
            'unit_id' => ['required', 'numeric'],
            'password' => ['required', 'string', 'min:8'], // Optional field
            'role_id' => ['required'], // Validate role input
        ]);

        $user = User::create($validated);
        $user->password = Hash::make($request->password);
    }
}

```

```

        $user->active = 1;
        $user->save();

        $user->syncRoles($request->input('role_id'));
        $user->assignRole($request->input('role_id'));
        return redirect()->route('users.index');
    }
    public function show(User $user)
    {
        return view('users.show', compact('user'));
    }
    public function edit(User $user)
    {
        $location = Location::all();
        $role = Role::all();
        $rank = Rank::all();
        $unit = Unit::all();
        return view('users.edit', compact('user','location','role','rank','unit'));
    }
    public function update(Request $request, User $user)
    {
        $user_detail = $request->validate([
            'service_no' => ['required', 'string'],
            'name' => 'required|string|max:255',
            'email' => 'required|string|max:255',
            'location_id' => ['required', 'numeric'],
            'rank_id' => ['required', 'numeric'],
            'unit_id' => ['required', 'numeric'],
            'password' => 'required|string|min:8', // Optional field
            'role_id' => ['required'], // Validate role input
        ]);
        $user->update($user_detail);
        $role = Role::findOrFail($request->input('role_id'));
        // Sync the roles for the user (in case role is updated)
        $user->syncRoles([$role->name]);
        $user->assignRole($role->name);

        $user->syncPermissions($request->input('permissions'));

        // $user->update($request->all());
        return redirect()->route('users.index');
    }
    public function destroy(User $user)
    {
        $user->delete();
        return redirect()->route('users.index');
    }
}

```

- How to create crud?

crud

ProductController

<?php

```
namespace App\Http\Controllers;

use App\Models\Product;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    public function index()
    {
        $products = Product::all();
        return view('products.index', compact('products'));
    }

    public function create()
    {
        return view('products.create');
    }

    public function store(Request $request)
    {
        Product::create($request->all());
        return redirect()->route('products.index');
    }

    public function show(Product $product)
    {
        return view('products.show', compact('product'));
    }

    public function edit(Product $product)
```



```

{
    return view('products.edit', compact('product'));
}

public function update(Request $request, Product $product)
{
    $product->update($request->all());
    return redirect()->route('products.index');
}

public function destroy(Product $product)
{
    $product->delete();
    return redirect()->route('products.index');
}
}

```

web.php

```

use App\Http\Controllers\ProductController;
Route::resource('products', ProductController::class);

```

migration

<?php

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

```

```

return new class extends Migration
{

```

```

/**
 * Run the migrations.
 */
public function up(): void
{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('category');
        $table->decimal('price', 8, 2);
        $table->integer('quantity');
        $table->timestamps();
    });
}

/**
 * Reverse the migrations.
 */
public function down(): void
{
    Schema::dropIfExists('products');
}
};

model
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\SoftDeletes;

```

```
use Illuminate\Database\Eloquent\Model;
```

```
class Product extends Model
```

```
{  
    use SoftDeletes;  
  
    protected $fillable = [  
        'name',  
        'category',  
        'price',  
        'quantity',  
        // Add any other fields you need  
    ];  
}
```

```
add_deleted_at row migration  
<?php
```

```
use Illuminate\Database\Migrations\Migration;  
use Illuminate\Database\Schema\Blueprint;  
use Illuminate\Support\Facades\Schema;
```

```
return new class extends Migration  
{  
    /**  
     * Run the migrations.
```

```

    */
    public function up(): void
    {
        Schema::table('products', function (Blueprint $table) {
            $table->softDeletes();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('products', function (Blueprint $table) {
            //
        });
    }
};

```

index.blade.php

```
@extends('adminlte::page')
```

```
@section('content')
```

```
<div class="container mt-5">
```

```
<div class="row justify-content-center">
```

```
<div class="col-md-10">
```

```
    @if (session('success'))
```

```
        <div class="alert alert-success">{{ session('success')

```

```
}}</div>
```

```

    @endif
{{--
    @if (session('status'))
        <div class="alert alert-success">{{ session('status')
    }}</div>
    @endif --}}
<div class="card mt-3">
    <div class="card-header">{{ __('Products') }}</div>

    <div class="card-body">
        <a href="{{ route('products.create') }}" class="btn btn-primary float-end">Add New Product</a>

        <table class="table table-bordered table-striped">
            <thead>
                <tr>
                    <th>Name</th>
                    <th>Category</th>
                    <th>Price</th>
                    <th>Quantity</th>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody>
                @foreach ($products as $product)
                    <tr>
                        <td>{{ $product->name }}</td>
                        <td>{{ $product->category }}</td>
                        <td>{{ $product->price }}</td>
                        <td>{{ $product->quantity }}</td>

```

```

        <td>
            <a href="{{ route('products.edit',
$product->id) }}" class="btn btn-xs btn-warning" data-
toggle="tooltip" title="Edit User" ><i class="fa fa-
pen"></i></a>

            <form action="{{ route('products.destroy',
$product->id) }}" method="POST" style="display:inline;"
onsubmit="return confirmDelete()" >
                @csrf
                @method('DELETE')
                <button type="submit" class="btn btn-
xs btn-danger" data-toggle="tooltip" title="Delete User" ><i
class="fa fa-trash"></i></button>
            </form>
            <a href="{{ route('products.show',
$product->id) }}" class="btn btn-xs btn-info" data-
toggle="tooltip" title="Edit User" ><i class="fa fa-eye"></i></a>
        </td>
    </tr>
    @endforeach
</tbody>
</table>

</div>
</div>
</div>
@endsection
<script>
    function confirmDelete(){

```

```
        return confirm('Are You sure you want to delete this  
record? This action be undone.');
```

```
    }
```

```
</script>
```

```
create.blade.php
```

```
@extends('adminlte::page')
```

```
@section('content')
```

```
<div class="container mt-5">
```

```
    <div class="row justify-content-center">
```

```
        <div class="col-md-7">
```

```
            @if (session('status'))
```

```
                <div class="alert alert-success">{{ session('status')  
}}</div>
```

```
            @endif
```

```
            <div class="card">
```

```
                <div class="card-header">{{ __(' Add Product')  
}}</div>
```

```
                <div class="card-body">
```

```
                    <form action="{{ route('products.store') }}"  
method="POST">
```

```
                        @csrf
```

```
                        <div class="mb-3">
```

```
                            <label for="">Name:</label>
```

```
                            <input type="text" name="name" required  
class="form-control"/>
```

```
                        </div>
```

```

        <div class="mb-3">
            <label for="">Category: </label>
            <input type="text" name="category" required
class="form-control"/>
        </div>
        <div class="mb-3">
            <label for="">Price: </label>
            <input type="number" name="price" required
class="form-control"/>
        </div>
        <div class="mb-3">
            <label for="">Quantity: </label>
            <input type="number" name="quantity"
required class="form-control"/>
        </div>
        <div class="mb-3">
            <button type="submit" class="btn btn-
primary">Save</button>
        </div>

    </form>
</div>
</div>
</div>
</div>
edit.blade.php
@extends('adminlte::page')

@section('content')

```



```

<div class="container mt-5">
  <div class="row justify-content-center">
    <div class="col-md-7">

      @if (session('status'))
        <div class="alert alert-success">{{ session('status')
      }}</div>
      @endif
      <div class="card">
        <div class="card-header">{{ __(' Edit Product')
      }}</div>
        <div class="card-body">
          <form action="{{ route('products.update', $product-
>id) }}" method="POST">
            @csrf
            @method('PUT')
            <div class="mb-3">
              <label for="">Name:</label>
              <input type="text" name="name" id="name"
class="form-control" value="{{ old('name', $product->name) }}"
required>
            </div>
            <div class="mb-3">
              <label for="">Category: </label>
              <input type="text" name="category" id="name"
class="form-control" value="{{ old('name', $product->name) }}"
required>
            </div>
            <div class="mb-3">
              <label for="">Price: </label>

```

```
        <input type="number" name="price" id="price"
class="form-control" value="{{ old('price', $product->price) }}"
step="0.01" required>
```

```
    </div>
```

```
    <div class="mb-3">
```

```
        <label for="">Quantity: </label>
```

```
        <input type="number" name="quantity"
id="quantity" class="form-control" value="{{ old('quantity',
$product->quantity) }}" required>
```

```
    </div>
```

```
    <div class="mb-3">
```

```
        <button type="submit" class="btn btn-
primary">Update</button>
```

```
    </div>
```

```
    </form>
```

```
    </div>
```

```
    </div>
```

```
    </div>
```

```
    </div>
```

```
</div>
```

```
show.blade.php
```

```
@extends('adminlte::page')
```

```
@section('content')
```

```
<div class="container mt-5">
```

```
    <div class="row justify-content-center">
```

```
        <div class="col-md-7">
```

```

    @if (session('status'))
        <div class="alert alert-success">{{ session('status')
    }}</div>
    @endif
    <div class="card">
        <div class="card-header"><strong>{{ $product->name
    }}</strong>
        </div></div>
        <div class="card-body">
            <ul class="list-group">
                <li class="list-group-item">
                    <strong>Category:</strong> {{ $product-
>Category }}
                </li>
                <li class="list-group-item">
                    <strong>Price:</strong> ${{
number_format($product->price, 2) }}
                </li>
                <li class="list-group-item">
                    <strong>Quantity:</strong> {{ $product-
>quantity }}
                </li>
                <li class="list-group-item">
                    <strong>Created At:</strong> {{ $product-
>created_at->format('d-m-Y H:i') }}
                </li>
                <li class="list-group-item">
                    <strong>Last Updated:</strong> {{ $product-
>updated_at->format('d-m-Y H:i') }}

```

```
</li>
</ul>
</div>
</div>
</div>
</div>
</div>
```

- How to bycrypt password?
 - Php artisan tinker
 - >Hash::make('yourPasswordHere');



Sample Project

<https://github.com/Samadhi-perera/ems.git>